CSE 25 - Introduction to Artificial Intelligence

Week 4 Thursday: Review

Learning Objectives:

Here's what we've worked towards in this course so far:

- Identify examples of artificial intelligence appearing in everyday situations described in the article and class discussion, including less obvious cases.
- Explain, using examples from NY Times article and discussion, why it can be unclear what should or should not be considered AI.
- Articulate what makes a system intelligent using examples discussed in class.
- Explain the Five Big Ideas of AI (**Perception**, **Representation**, **Learning**, **Interaction**, **Societal Impact**) and apply them to multiple systems, noting where they do and do not apply.
- Identify major areas where AI techniques are commonly used (such as vision, language, recommendation systems, robotics, and multimodal systems) and explain what kinds of problems they address.
- Interpret CSE course titles and descriptions to make informed, high-level connections between AI ideas, application areas, and the curriculum.
- Describe Turing's "Imitation Game" and its connection to artificial intelligence.
- Identify and describe the key stages of a machine learning pipeline: **Problem formulation**, **Data/Experience collection**, **Data processing**, **Training**, **Evaluation**, **Deployment**.
- Explain why machine learning systems are iterative.
- Classify learning systems as **supervised**, **unsupervised**, or **reinforcement** learning based on the type of data or feedback used.
- Explain how a simple linear model makes predictions using parameters.
- Compute and interpret pointwise and total error.
- Describe brute-force parameter search and when it is feasible.
- Explain why error minimization is central to learning systems.
- Describe several ways of measuring error of a model and compare their properties.
- Interpret error as a function of a model parameter.
- Explain how changing a parameter affects error.
- Describe brute-force search as a way to find better parameters.
- Explain, at a high level, how slope (rate of change) tells us which direction to adjust a parameter.
- Describe the difference between local and global optima.

- Connect these ideas to the motivation behind gradient descent.
- Define simple and multiple linear regression as fitting a hyperplane to predict target values from input features.
- Distinguish between **regression** and **classification** problems
- Explain how a **perceptron** uses a weighted sum and bias to make decisions
- Describe how the perceptron updates its parameters when it makes a mistake
- Explain what the words **features**, **labels**, **bias**, **decision boundary**, and **epoch** mean
- Define linear separability and give examples of data sets that are or aren't linearly separable
- Distinguish between local and global minima and explain how an algorithm might avoid local minima.
- Use a trained linear model to make predictions on new, unseen data
- Compute **accuracy** from predictions and labels (or class)
- Use a **confusion matrix** to compute **precision**, **recall**, and **F1** score
- Connect metrics of classification to goals for applying ML systems
- Describe why the perceptron update rule works for binary settings.
- Define perceptron loss.
- Explain the purpose of **train / validation / test** splits
- Define and describe **generalization**, **overfitting**, and **underfitting**
- Interpret a **training** curve
- Interpret a **learning** curve

Today we'll review some of these key concepts and explore new examples.

Instructions

Use your copy of this notebook on Datahub and complete it during class. Work through the cells below **in order**. You may discuss with your neighbors, but make sure you understand each step yourself.

SUBMISSION: When finished, download it as a PDF and upload it to Gradescope under `In-Class - Week 4 Thursday` to receive credit. To download it as a PDF, on DataHub go to `File -> Save and Export Notebook As -> PDF`.

Big ideas in AI and ML pipeline

Our running example today will be that of **weather prediction**.

Describe how each of the Five Big Ideas of AI might apply to a weather prediction system.

`Perception`:

`Representation`:

`Learning`:

`Interaction`:

`Societal Impact`:

Describe the type of data or feedback that could be used in a system related to **weather prediction** that lends itself to each of the machine learning paradigms we discussed:

`Supervised Learning`:

`Unsupervised Learning`:

`Reinforcement Learning`:

Regression

For input x, the model with parameters weight w and bias b predicts the value

$$wx + b$$

Using a linear model to predict values, and calculating error in the prediction

Trace the code in the cells below and add comments.

In [ ]:
```
def get_predictions_v2(input_values, w, b):
    '''
    input_values: list of input values
    w: weight (slope)
    b: bias (intercept)

    Complete the function that calculates the predicted output values using
    Return a list of predicted values.
    '''
    predicted_values = []
    for input_value in input_values:
        predicted_values.append(input_value * w + b)
    return predicted_values

def get_pointwise_error(actual_values, predicted_values):
    '''
    actual_values: list of actual kilometer values
    predicted_values: list of predicted kilometer values

    Complete the function that calculates the error (actual minus predicted)
    '''
    return [actual - predicted for actual, predicted in zip(actual_values, p

def get_pointwise_absolute_error(actual_values, predicted_values):
    '''
```

```
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the absolute error between predict
    '''
    return [abs(actual - predicted) for actual, predicted in zip(actual_valu

def get_total_absolute_error(actual_values, predicted_values):
    '''
    errors: list of errors for each data point
    Complete the function that calculates the total error by summing the val
    '''
    return sum(get_pointwise_absolute_error(actual_values,predicted_values))

def get_mean_absolute_error(actual_values, predicted_values):
    '''
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the Mean Absolute Error (MAE) betw
    Return the MAE.
    '''
    return  (sum(get_pointwise_absolute_error(actual_values, predicted_value

def get_mean_squared_error(actual_values, predicted_values):
    '''
    actual_values: list of actual output values
    predicted_values: list of predicted output values

    Complete the function that calculates the Mean Squared Error (MSE) betwe
    Return the MSE.
    '''
    sq_errors = []
    for act, pred in zip(actual_values, predicted_values):
        sq_errors.append((act-pred)**2)
    return sum(sq_errors)*1.0/len(sq_errors)
```

In [ ]:

```
# Functions for plotting

# May need to run pip install plotly
import plotly.graph_objects as go

def plot_error_surface(feat1,feat2,errors,title):
    fig_mse = go.Figure(data=[go.Scatter3d(
        x=feat1,
        y=feat2,
        z=errors,
        mode='markers',
        marker=dict(
            size=5,
            color=errors,
            colorscale='Viridis',
            colorbar=dict(title='MSE')
        )
    )])
```

```
    best_idx = errors.index(min(errors))

    fig_mse.add_trace(go.Scatter3d(
        x=[feat1[best_idx]],
        y=[feat2[best_idx]],
        z=[errors[best_idx]],
        mode='markers',
        marker=dict(size=10, color='red'),
        name='Best Parameters'
    ))
    fig_mse.update_layout(
        scene=dict(
            xaxis_title='Weight',
            yaxis_title='Bias',
            zaxis_title='MSE'
        ),
        title=f'MSE Surface: {title}',
        width=1000,  # Increase width
        height=800   # Increase height
    )
    fig_mse.show()
```

Using brute-force search to train model for regression

Trace the code in the cells below and add comments.

In [ ]:

```
import matplotlib.pyplot as plt

def brute_force_search(X, y, error_function, lower_w = 0.0, upper_w = 1.0, l
    best_weight = None
    best_bias = None
    lowest_error = float('inf')

    weights = []
    bias = []
    error_list = []

    w = lower_w
    b = lower_b

    while w <= upper_w:
        while b <= upper_b:
            predicted = get_predictions_v2(X, w, b)
            error = error_function(y, predicted)
            error_list.append(error)
            weights.append(w)
            bias.append(b)
            if error < lowest_error:
                lowest_error = error
                best_weight = w
                best_bias = b
            b += step_size_b
        b = lower_b
        w += step_size_w
```

```
    print(f'Best weight (conversion factor): {best_weight:.4f}')
    print(f'Best bias: {best_bias:.4f}')
    print(f'Lowest error: {lowest_error:.2f}')

    return weights, bias, error_list
```

In [ ]:
```
# Example data set for learning conversion from degrees Fahrenheit to degree

fahrenheit = [32, 50, 68, 86, 104] # x-axis values (INPUT DATA)
celsius = [0, 10, 20, 30, 40] # y-axis values (OUTPUT DATA)

print("Brute force search with total absolute error")
brute_force_search(fahrenheit, celsius, get_total_absolute_error)

print("\nBrute force search with mean absolute error")
brute_force_search(fahrenheit, celsius, get_mean_absolute_error)

print("\nBrute force search with mean squared error")
weights_bf, biases_bf, mse_bf = brute_force_search(fahrenheit, celsius, get_
```

Q: Why do all three error functions give the same best model parameters?

`YOUR ANSWER HERE`

Q: Why is the lowest error different for each of the error functions even though the model parameters are the same?

`YOUR ANSWER HERE`

Using gradient descent to train model for regression

Trace the code in the cells below and add comments.

In [ ]:
```
def get_gradients_w_b(error_function, input_values, actual_values, w, b, eps
    """
    Approximates the partial derivatives of the error with respect to w and
    using finite differences.

    Returns:
        (grad_w, grad_b)
    """
    predicted = get_predictions_v2(input_values,w,b)
    error = error_function(actual_values,predicted)

    predicted_w = get_predictions_v2(input_values, w+epsilon, b)
    error_w = error_function(actual_values,predicted_w)
    grad_w = (error_w - error)/epsilon

    predicted_b = get_predictions_v2(input_values,w,b+epsilon)
    error_b = error_function(actual_values,predicted_b)
    grad_b = (error_b-error)/epsilon

    return grad_w, grad_b
```

```
def gradient_descent_w_b(X, y, initial_w = 1, initial_b = -60,  learning_rat

    w = initial_w
    b = initial_b

    grad_weights = []
    grad_biases = []
    grad_errors = []

    for step in range(num_steps):
        predicted = get_predictions_v2(X, w, b)
        error = get_mean_squared_error(y, predicted)
        grad_weights.append(w)
        grad_biases.append(b)
        grad_errors.append(error)
        grad_w, grad_b = get_gradients_w_b(get_mean_squared_error, X, y, w,
        w = w - learning_rate * grad_w
        b = b - learning_rate * grad_b
        if step % 25000 == 0:
            print(f"Step {step}: w = {w:.4f}, b = {b:.4f}, error = {error:.4

    print(f"Best weight (w) found: {w:.4f}")
    print(f"Best bias (b) found: {b:.4f}")
    print(f"Mean squared error at best parameters: {grad_errors[-1]:.4f}")

    return grad_weights, grad_biases, grad_errors
```

In [ ]:
```
print("Gradient descent to train model for degrees Fahrenheit to degrees Cel
weights_gd, biases_gd, mse_gd = gradient_descent_w_b(fahrenheit, celsius)
```

In [ ]:
```
plot_error_surface(weights_bf,biases_bf,mse_bf,"Brute-force search")
plot_error_surface(weights_gd,biases_gd,mse_gd,"Gradient descent")
```

Q: Which performed better, brute-force search or gradient descent?

`YOUR ANSWER HERE`

Q: Is this a fair comparison?

`YOUR ANSWER HERE`

Binary classification

For input $x = [x1, \ldots xn]$, the perceptron with parameters weights w and bias b predicts the label $+1$ if the score

$$z = \sum_{i=1}^{n} w_i x_i + b$$

is greater than or equal to 0, and predicts the label -1 if $z < 0$.

Using a perceptron model to predict labels, and calculating error in the prediction

Trace the code in the cells below and add comments.

In [ ]:
```
def perceptron_predict(x, w, b):
    '''
    x: 2D input list [x1, x2]
    w: 2D weight list [w1, w2]
    b: bias term
    '''
    score = w[0]*x[0]+ w[1]*x[1] +b
    if score >= 0:
        return 1
    else:
        return -1

def perceptron_predict_all(X_list, w, b):
    '''
    Return a list of predictions (+1 or -1) for every point in X_list.
    '''
    return [1 if w[0]*x[0] + w[1]*x[1] + b >= 0 else -1 for x in X_list]

def compute_metrics(y_true, y_pred):
    '''
    Compute accuracy, precision, and recall for binary labels in {-1, +1}.

    Treats +1 as the positive class.

    Returns a tuple (accuracy, precision, recall).
    Precision and recall are set to 0.0 when the denominator would be zero
    (i.e. the model never predicts positive, or there are no true positives)
    '''
    tp = sum(1 for t, p in zip(y_true, y_pred) if t ==  1 and p ==  1)
    tn = sum(1 for t, p in zip(y_true, y_pred) if t == -1 and p == -1)
    fp = sum(1 for t, p in zip(y_true, y_pred) if t == -1 and p ==  1)
    fn = sum(1 for t, p in zip(y_true, y_pred) if t ==  1 and p == -1)

    accuracy  = (tp + tn) / len(y_true)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall    = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1        = 2.0 * (precision * recall)/(precision + recall) if (precisic

    return accuracy, precision, recall, f1

def perceptron_loss(y, z):
    '''
    Computes the perceptron loss for a given example to a model
    '''
    return max(0, -y*z)
```

In [ ]:
```
import random

def perceptron_train_eval_with_history(X, y, num_epochs=10,X_test=None, y_te
```

```
    '''
    Train a perceptron and record loss, accuracy, precision, and recall
    after every individual data point is processed (across all epochs).

    X: list of [x1, x2] feature pairs
    y: list of labels in {-1, +1}
    num_epochs: number of full passes over the data
    X_test, y_test: optional test sets for calculating test accuracty

    Returns
    w, b : final weights and bias
    history : dictionary with keys 'loss', 'accuracy', 'precision', 'recall'
              each a list of length (num_epochs * len(X)). Also has a key 't
    '''
    w = [0.0, 0.0]
    b = 0.0

    history = {'loss': [], 'accuracy': [], 'precision': [], 'recall': [], 'f

    rng = random.Random(42)

    for epoch in range(num_epochs):
        combined = list(zip(X, y))
        rng.shuffle(combined)

        for x, y_true in combined:
            score  = w[0] * x[0] + w[1] * x[1] + b
            y_pred = 1 if score >= 0 else -1

            if y_pred != y_true:
                w[0] += y_true * x[0]
                w[1] += y_true * x[1]
                b    += y_true

            all_preds = perceptron_predict_all(X, w, b)

            total_loss = sum(
                max(0.0, -yt * (w[0]*xt[0] + w[1]*xt[1] + b))
                for xt, yt in zip(X, y)
            )

            acc, prec, rec, f1 = compute_metrics(y, all_preds)

            history['loss'].append(total_loss)
            history['accuracy'].append(acc)
            history['precision'].append(prec)
            history['recall'].append(rec)
            history['f1'].append(f1)

        if X_test is not None and y_test is not None:
            test_preds = perceptron_predict_all(X_test, w, b)
            test_acc, _, _ , _ = compute_metrics(y_test, test_preds)
            history['test_accuracy'].append(test_acc)

        if history['accuracy'][-1] == 1.0: break
```

```
    return w, b, history
```

In [ ]:

```
# With stratification
from typing import List, Sequence, Tuple

def train_test_split_v2(
    X: Sequence[Sequence[float]],
    y: Sequence[int],
    test_ratio: float = 0.2,
    seed: int = 0,
    shuffle: bool = True,
) -> Tuple[List[List[float]], List[List[float]], List[int], List[int]]:
    '''
    Split data set with stratification: preserves class proportions in each

    X: list of input features
    y: list of target labels
    test_ratio: fraction of the data to put in the test set. Must be betweer
    seed: Random seed used for shuffling, so results are reproducible. Defau
    shuffle: If True, randomly shuffle the data before splitting.
            If False, take the first (1 - test_ratio) fraction for training
    '''
    # Create a local instance random number generator, in case shuffle is Tr
    rng = random.Random(seed)
    # Group indices by class label
    labels = list(set(y))
    class_indices = {label: [i for i, v in enumerate(y) if v == label] for l

    train_indices = []
    test_indices  = []

    for label in labels:
        idxs = class_indices[label][:]
        if shuffle:
            rng.shuffle(idxs)
        n_test = max(1, round(len(idxs) * test_ratio))
        test_indices  += idxs[:n_test]
        train_indices += idxs[n_test:]

    if shuffle:
        rng.shuffle(train_indices)
        rng.shuffle(test_indices)

    X_list = [list(row) for row in X]
    y_list = [int(v)    for v in y]

    return (
        [X_list[i] for i in train_indices],
        [X_list[i] for i in test_indices],
        [y_list[i] for i in train_indices],
        [y_list[i] for i in test_indices],
    )
```

In [ ]:

```python
# Functions for plotting

import numpy as np

def combined_perceptron_plot(w1=0, w2=0, b=0, X_data = None, y_data=None):
    '''
    Display two views of data set and perceptron model performance.

    w1,w2: model weights (default: 1.0)
    b: model bias (default: -2.0)
    '''
    if X_data is None or y_data is None:
        print("No data provided.")
        return

    # Compute data-driven axis limits
    x1_vals = [x[0] for x in X_data]
    x2_vals = [x[1] for x in X_data]
    PAD = 0.10   # 10 % padding on each side

    x1_min, x1_max = min(x1_vals), max(x1_vals)
    x2_min, x2_max = min(x2_vals), max(x2_vals)
    x1_pad = (x1_max - x1_min) * PAD or 1.0   # guard against zero range
    x2_pad = (x2_max - x2_min) * PAD or 1.0

    feat_x1_lo, feat_x1_hi = x1_min - x1_pad, x1_max + x1_pad
    feat_x2_lo, feat_x2_hi = x2_min - x2_pad, x2_max + x2_pad

    # Score range: compute the actual score for every data point, then add p
    scores = [w1 * x[0] + w2 * x[1] + b for x in X_data]
    z_lo = min(scores) - abs(min(scores)) * PAD - 1.0
    z_hi = max(scores) + abs(max(scores)) * PAD + 1.0

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # LEFT: 1D Step View (sign)
    n_steps = 400
    z_axis = [z_lo + (z_hi - z_lo)*i/n_steps for i in range(n_steps + 1)]
    ax1.plot(z_axis, [1 if z >= 0 else -1 for z in z_axis],
             color='green', drawstyle='steps-mid', label="sign(z)")
    ax1.axvline(x=0, color='black', linestyle='--', alpha=0.5)

    total_loss = 0  # Initialize loss counter

    for i in range(len(X_data)):
        x1, x2 = X_data[i]
        y = y_data[i]
        z_score = w1 * x1 + w2 * x2 + b
        pred = 1 if z_score >= 0 else -1

        loss = max(0.0, -y * z_score)  # Perceptron loss
        total_loss += loss

        color, m = ('blue', 'x') if y == 1 else ('red', 'o')
        if y == 1:
```

```
            ax1.scatter(z_score, pred, color=color, marker=m, s=100, linewic
        else:
            ax1.scatter(z_score, pred, color=color, marker=m, s=80, edgecolc

    # Update title with the calculated loss
    ax1.set_title(f"Perceptron: Score vs Prediction\nTotal Perceptron Loss:
    ax1.set_xlabel("Score (z)")
    ax1.set_ylim(-1.1, 1.1)
    ax1.set_yticks([-1, 1])

    # RIGHT: 2D Feature Space (Speedy Rendering)
    xx, yy = np.meshgrid(np.linspace(feat_x1_lo, feat_x1_hi, 200),
                         np.linspace(feat_x2_lo, feat_x2_hi, 200))
    zz = w1 * xx + w2 * yy + b

    # Binary background: everything <0 is blue, everything >0 is red
    ax2.contourf(xx, yy, zz, levels=[-1e9, 0, 1e9], colors=['red','blue'], a

    if w2 != 0:
        x_line = np.array([feat_x1_lo, feat_x1_hi])
        y_line = -(w1 * x_line + b) / w2
        ax2.plot(x_line, y_line, "k-", linewidth=2)

    for i in range(len(X_data)):
        x1, x2 = X_data[i]
        y = y_data[i]
        if y == 1:
            ax2.scatter(x1, x2, color='blue', marker='x', s=100, linewidths=
        else:
            ax2.scatter(x1, x2, color='red', marker='o', s=80, edgecolors='k
        # label each point with its coordinates
        ax2.annotate(
        f'({x1:.2f}, {x2:.2f})',    # text to display
        xy=(x1, x2),              # point the text refers to
        xytext=(5, 5),            # offset in points from the marker (right
        textcoords='offset points',
        fontsize=10,
        color='black',
        )

    ax2.set_xlim(feat_x1_lo, feat_x1_hi)
    ax2.set_ylim(feat_x2_lo, feat_x2_hi)
    ax2.set_title("2D View: Hard Decision Regions")
    plt.tight_layout()
    plt.show()

def perceptron_metrics_plot(X_tr, history, w, b):
    num_epochs_run = len(history['test_accuracy'])
    steps = list(range(1, len(history['accuracy']) + 1))
    n_pts = len(X_tr)

    print(f"-----\nTracking metrics after perceptron training")
    print(f"Number of training data points: {n_pts}, num_epochs_run: {num_ep

    # Vertical lines mark the end of each epoch
    epoch_boundaries = [n_pts * e for e in range(1, num_epochs_run + 1)]
```

```
    # Five-panel figure
    metrics = [
    ('loss',      'Total Perceptron Loss\n(sum over training set)', 'tab:rec
    ('accuracy',  'Accuracy',                                       'tab:blue'),
    ('precision', 'Precision',                                      'tab:green'),
    ('recall',    'Recall',                                         'tab:orange'),
    ('f1',        'F1',                                             'tab:purple'),
    ]

    fig, axes = plt.subplots(5, 1, figsize=(10, 12), sharex=True)

    for ax, (key, ylabel, color) in zip(axes, metrics):
        ax.plot(steps, history[key], color=color, linewidth=1.5)
        for eb in epoch_boundaries:
            ax.axvline(x=eb, color='gray', linestyle='--', linewidth=0.8, al
        ax.set_ylabel(ylabel, fontsize=10)
        ax.set_ylim(-0.05, max(history[key]) * 1.15 + 0.01)
        ax.grid(True, alpha=0.4)

    # Label epoch boundaries on the top panel only
    for i, eb in enumerate(epoch_boundaries):
        axes[0].text(eb, axes[0].get_ylim()[1] * 0.95,
                 f'end\nepoch {i+1}', ha='right', fontsize=7, color='gray')

    axes[-1].set_xlabel('Data points processed (across all epochs)', fontsiz
    fig.suptitle(
        f'Per-Point Training Curves for Perceptron',
        fontsize=13, y=1.01)
    plt.tight_layout()
    plt.show()

    print(f"Final weights: w = {[round(wi, 3) for wi in w]}, b = {round(b, 3

def perceptron_train_curve(X_tr, history):
    n_pts = len(X_tr)
    num_epochs_run = len(history['test_accuracy'])
    epochs = list(range(1,num_epochs_run+1))

    # Sample epoch-end values from the per-point history
    epoch_train_err = [1.0 - history['accuracy'][n_pts*(e+1)-1] for e in ran
    epoch_test_err  = [1.0 - a for a in history['test_accuracy']]

    epochs = list(range(1, num_epochs_run + 1))
    plt.figure(figsize=(8, 4))
    plt.plot(epochs, epoch_train_err, marker='o', label='Train error')
    plt.plot(epochs, epoch_test_err,  marker='s', label='Test error')
    plt.xlabel('Epoch')
    plt.xticks(epochs)
    plt.ylabel('Error rate')
    plt.title('Training Curve: Error per Epoch')
    plt.legend()
    plt.grid(True)
    print(f"-----\nPlotting training curve")
    plt.show()
```

In [ ]:
```
# Example data set for learning classification of rainy / not-rainy based on

measurements = [    [32, 90], [50, 85], [68, 80], [86, 75], [55, 70],
    [75, 50], [90, 40], [60, 35], [45, 30], [80, 25] ]
rainy = [+1, +1, +1, +1, +1,  -1, -1, -1, -1, -1]

from ipywidgets import interact, fixed

print(f"-----\nVisualizing perceptron score and classification")
interact(combined_perceptron_plot,
         w1=(-200, 200, 0.5),
         w2=(-200, 200, 0.5),
         b=(-20, 20, 0.1),
         X_data = fixed(measurements),
         y_data = fixed(rainy))

X_tr, X_tt, y_tr, y_tt = train_test_split_v2(measurements, rainy, test_ratio

w, b, history = perceptron_train_eval_with_history(X_tr, y_tr, X_test =X_tt,

perceptron_metrics_plot(X_tr, history, w, b)

perceptron_train_curve(X_tr, history)
```

Q: Identify the axes of each graph, along with key points in the graph. Connect these points to milestones in the training.

`YOUR ANSWER HERE`

We have covered two foundational linear models so far. In each case, given model parameters $\mathbf{w} = [\mathbf{w_1} \cdots \mathbf{w_n}]$ and bias b, and an input example $\mathbf{x} = [\mathbf{x_1} \cdots \mathbf{x_n}]$,

1. Compute a score:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

2. Turn the score into output using an **activation** function.

For each model, we use *training* to determine the model parameters, based on examples in the training set. The goal of training is to *mininmize* loss when predicting the target values for examples in the training set using the model.

- **Linear Regression**: Uses the identity activation function and is trained with Mean Squared Error (MSE) loss. This model is suitable for regression tasks where the output is continuous.

- **Perceptron**: Uses the sign (or step) activation function and is trained using the perceptron update rule, which can be interpreted as minimizing the perceptron loss. This model is designed for binary classification with hard decision boundaries.

Although these models differ in activation functions and loss functions, they share the same underlying structure: a single linear decision boundary.

What about nonlinear models?

The same concerns about *underfitting* and *overfitting* models apply to more complicated models.

We say a model **underfits** when it hasn't learned enough from the training data.

Signs of underfitting:

- Training error is **high** (the model makes mistakes even on examples it was trained on)
- Test error is also **high** (it may be close to training error, and both are bad)

We say a model **overfits** when it has learned the training examples *too specifically* — fitting the noise or quirks of the training set rather than the underlying pattern.

Signs of overfitting:

- Training error is **low**
- Test error is **high** (much higher than training error)
- Training metric keeps improving with epochs, whereas validation metric improves, then gets worse

The goal is to find the "sweet spot" between underfitting and overfitting where both training error and test error are low.

Let's try to fit a **curve** to a small set of training data.

Use the **degree** slider to change how flexible the model is:

- Small degree -> simple curve (line, parabola, cubic, etc)
- Large degree -> very bendy curve

The model is trying to follow the pattern in the training points.

The polynomial degree is a *hyperparameter* that controls model capacity.

Try this:

1. Increase the degree until the curve looks very wiggly.
2. Turn on **show val data**.
3. Notice that the complicated curve may not match the validation points well.

Even though the model fits training data very closely, it may perform worse on new data.

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display

rng = np.random.default_rng(0)
n_train, n_val = 12, 60

x_train = np.sort(rng.uniform(0, 1, n_train))
y_train = np.sin(2*np.pi*x_train) + rng.normal(0, 0.25, n_train)

x_val = np.sort(rng.uniform(0, 1, n_val))
y_val = np.sin(2*np.pi*x_val) + rng.normal(0, 0.25, n_val)

x_plot = np.linspace(0, 1, 300)

def mse(y, yhat):
    return float(np.mean((np.asarray(y) - np.asarray(yhat))**2))


deg_slider = widgets.IntSlider(value=1, min=0, max=10, step=1, description="
show_true = widgets.Checkbox(value=False, description="show true curve")
show_val  = widgets.Checkbox(value=False, description="show val data")
out = widgets.Output()

def redraw(degree, show_true_curve, show_val_data):
    with out:
        out.clear_output(wait=True)

        coeffs = np.polyfit(x_train, y_train, deg=degree)

        yhat_train = np.polyval(coeffs, x_train)
        yhat_val   = np.polyval(coeffs, x_val)
        yhat_plot  = np.polyval(coeffs, x_plot)

        train_mse = mse(y_train, yhat_train)
        val_mse   = mse(y_val, yhat_val)

        plt.figure(figsize=(7,4))

        plt.scatter(x_train, y_train, label="train")

        # show validation points only if toggled
        if show_val_data:
            plt.scatter(x_val, y_val, alpha=0.35, label="val")

        plt.plot(x_plot, yhat_plot, linewidth=2, label=f"fit (deg={degree})"

        if show_true_curve:
            plt.plot(x_plot, np.sin(2*np.pi*x_plot), linestyle="--", label="

        if show_val_data:
            title = f"Train MSE: {train_mse:.3f} | Val MSE: {val_mse:.3f}"
        else:
            title = f"Train MSE: {train_mse:.3f}"
```

```
        plt.title(title)
        plt.ylim(-2, 2)
        plt.legend()
        plt.show()

ui = widgets.VBox([deg_slider, show_true, show_val, out])
display(ui)

widgets.interactive_output(
    redraw,
    {
        "degree": deg_slider,
        "show_true_curve": show_true,
        "show_val_data": show_val
    }
)
```

Q. In the above graph, which model degree would you choose based on the validation loss?

`YOUR ANSWER HERE`